

Introduction to Gradient Descent

From “what to minimize” to *how* to minimize it

Topic 6 of Core ML Concepts

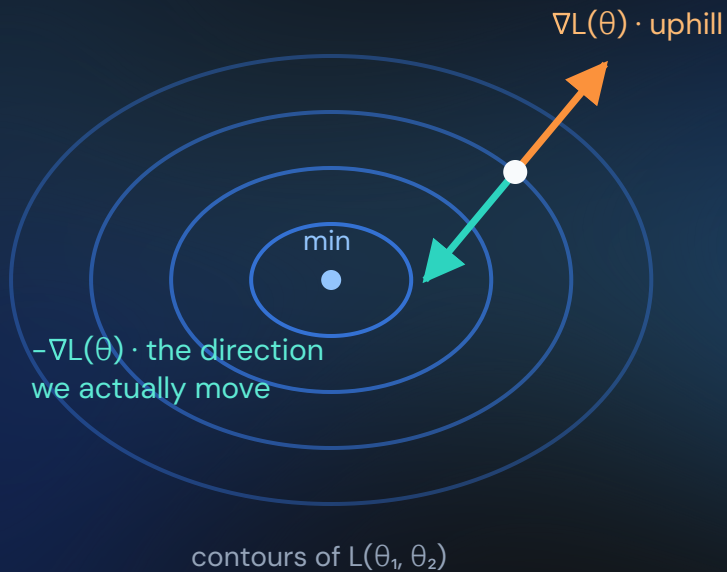
$$\arg \min_{\theta} \hat{R}(\theta)$$

→

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \ell(\theta)$$

The algorithm behind nearly every model in this course

The Gradient — Direction of Steepest Change



$$\nabla_{\theta} \ell(\theta) = \left[\frac{\partial \ell}{\partial \theta_1}, \frac{\partial \ell}{\partial \theta_2}, \dots, \frac{\partial \ell}{\partial \theta_k} \right]^T$$

One partial derivative per parameter, collected into a single vector — loss as a function of $\theta = (\theta_1, \dots, \theta_k)$.

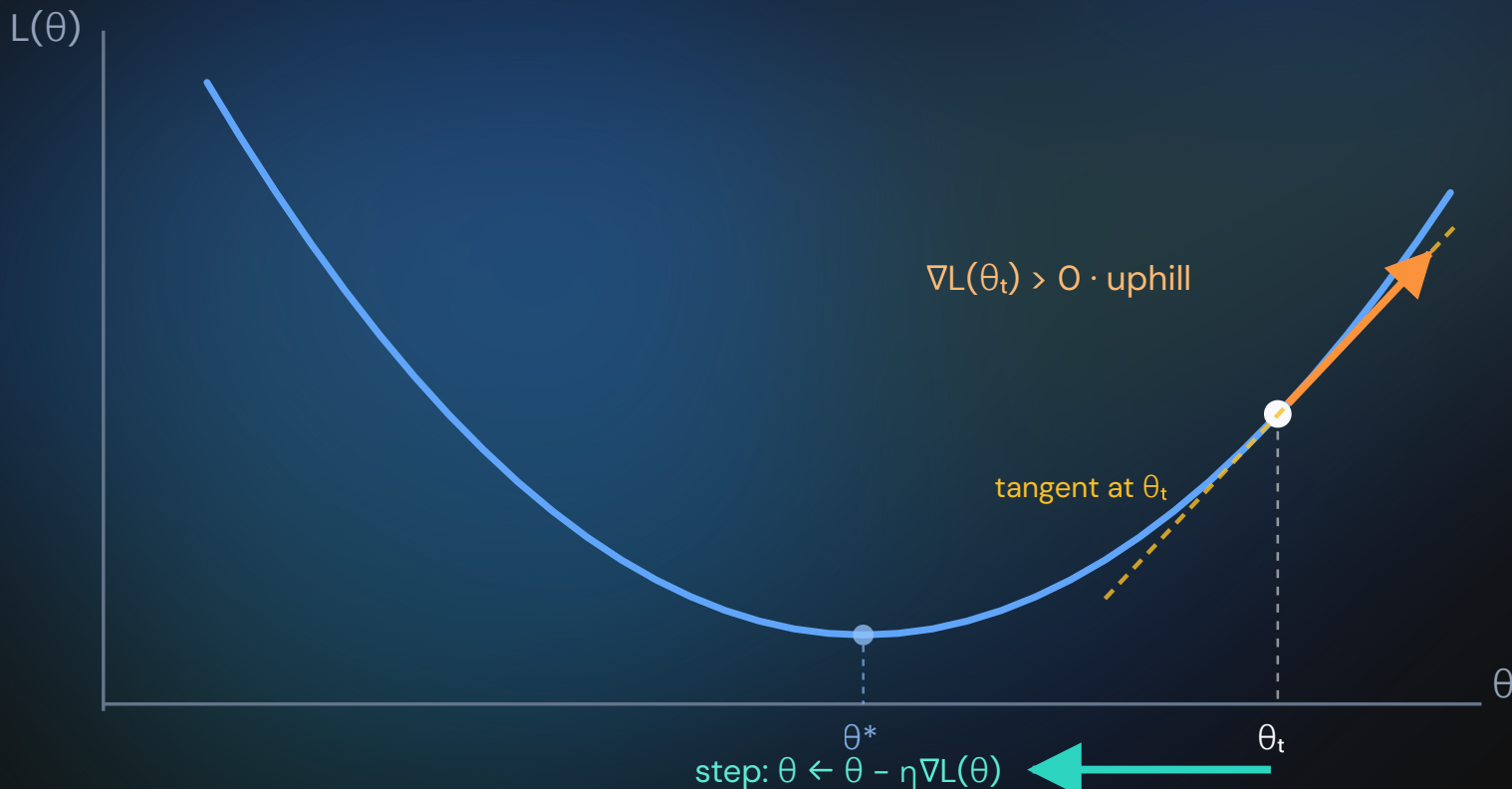
The gradient points in the direction of steepest **increase** at that point.

So its negative decreases the loss fastest, locally. That single fact is the whole algorithm.

This is the calculus lecture from Unit 1 (Mathematical Foundations) put directly to work.

One Parameter: the Gradient Is Just the Slope

One parameter $\Rightarrow$ the gradient is a single number, the slope. Positive slope $\Rightarrow$ decrease θ .



Same rule runs per-coordinate when θ is a vector.

The Gradient Descent Update Rule

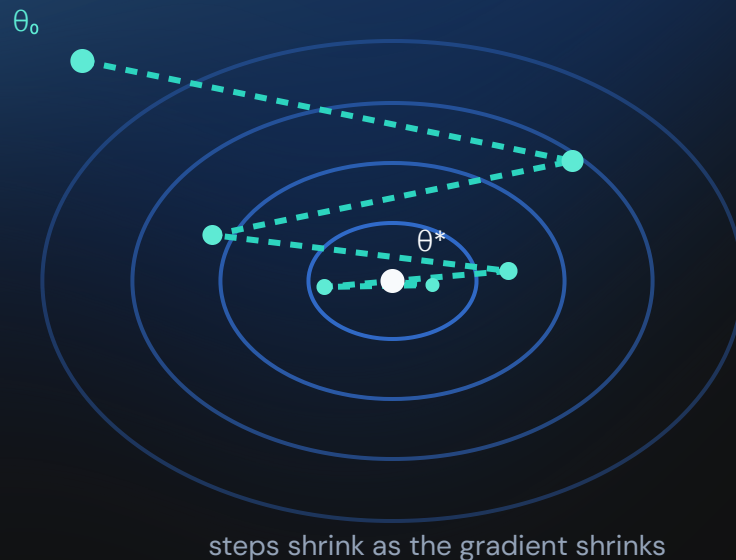
$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_{\theta} \ell(\theta^{(t)})$$

Initialize. Start from some θ_0 — often random, often just zeros.

Step. Repeatedly move in the negative gradient direction.

Learning rate η . Controls step size. A *hyperparameter* chosen by the practitioner, not learned from data.

Stop. When the loss stops decreasing meaningfully, or after a fixed number of iterations.



The Loop, in Code

```
import numpy as np

def gradient_descent(grad_fn, theta_init, lr=0.01, n_steps=1000):
    theta = theta_init.copy()
    for _ in range(n_steps):
        grad = grad_fn(theta)
        theta = theta - lr * grad
    return theta
```

The loop is loss-agnostic

`grad_fn` is the only thing that knows which loss is being minimized. Swap it and you have a different model — the loop never changes.

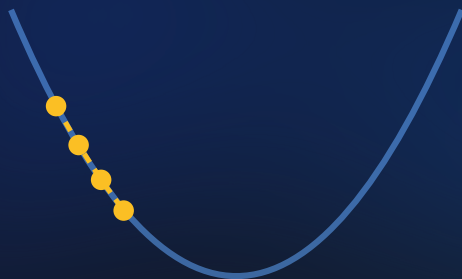
Everything else is a hyperparameter

`lr` and `n_steps` are chosen, not fitted. No amount of training data will tell you what they should be.

The rest of this lecture writes one `grad_fn` for each of three loss functions.

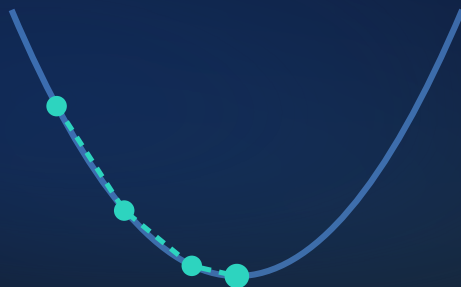
Choosing the Learning Rate

Same loss surface, same starting point, same update rule — only η changes.



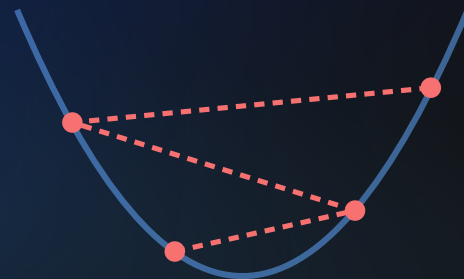
η too small

correct direction, glacial progress



η just right

converges in a few steps



η too large

overshoots, then diverges

The gradient supplies the **direction**; η alone decides **how far** to trust it. Nothing in the data picks η for you.

Deriving the Gradient of MSE · Scalar Form

Model and loss

$$f_{\theta}(x) = x^{\top} \theta, \quad \ell(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i - x_i^{\top} \theta)^2$$

Differentiate with respect to one parameter θ_j , using the chain rule twice: the square, then the residual.

Per-parameter partial derivative

$$\begin{aligned} \frac{\partial \ell}{\partial \theta_j} &= \frac{1}{n} \sum_{i=1}^n 2(y_i - x_i^{\top} \theta) \cdot (-x_{ij}) \\ &= -\frac{2}{n} \sum_{i=1}^n (y_i - x_i^{\top} \theta) x_{ij} \end{aligned}$$

The step everyone drops: $\frac{\partial}{\partial \theta_j} (y_i - x_i^{\top} \theta) = -x_{ij}$ — the **minus sign** comes from the residual, not the square.

MSE Gradient · Vectorized and in Code

$$\nabla_{\theta} \ell(\theta) = \frac{2}{n} X^T (X\theta - y)$$

```
import numpy as np

def mse_grad(theta, X, y):
    n = X.shape[0]
    residual = X @ theta - y
    return (2 / n) * X.T @ residual
```

Note the residual is written `X @ theta - y`, which absorbs the minus sign from the scalar form.

scalar form

$\sum_i$ over n rows

$-(y_i - x_i^T \theta)$

x_{ij}

vectorized

$X^T (\cdot)$

$X\theta - y$

column j of X

k scalars stacked = one gradient vector

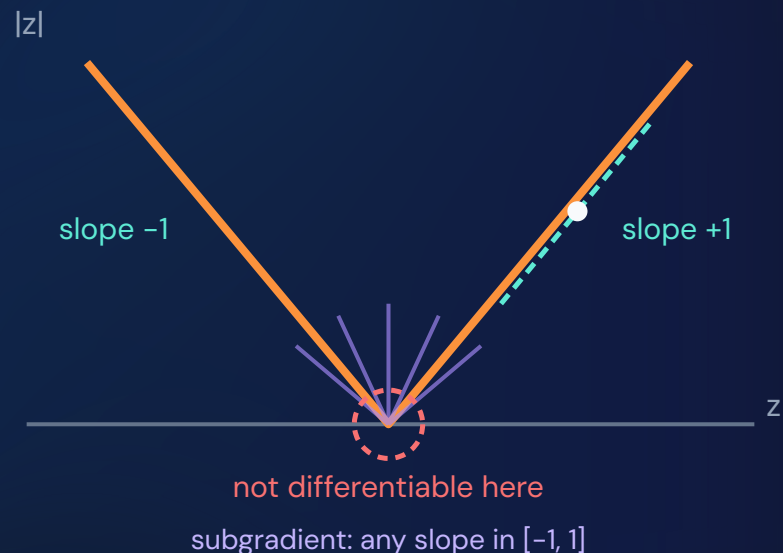
Deriving the Gradient of MAE

$$\ell(\theta) = \frac{1}{n} \sum_{i=1}^n |y_i - x_i^\top \theta|$$

$$\frac{\partial \ell}{\partial \theta_j} = -\frac{1}{n} \sum_{i=1}^n \text{sign}(y_i - x_i^\top \theta) x_{ij}$$

```
import numpy as np
```

```
def mae_grad(theta, X, y):  
    n = X.shape[0]  
    residual = y - X @ theta  
    # sign(0) is a subgradient choice; np.sign returns 0  
    return -(1 / n) * X.T @ np.sign(residual)
```



Logistic Loss · Setup and the Sigmoid Model

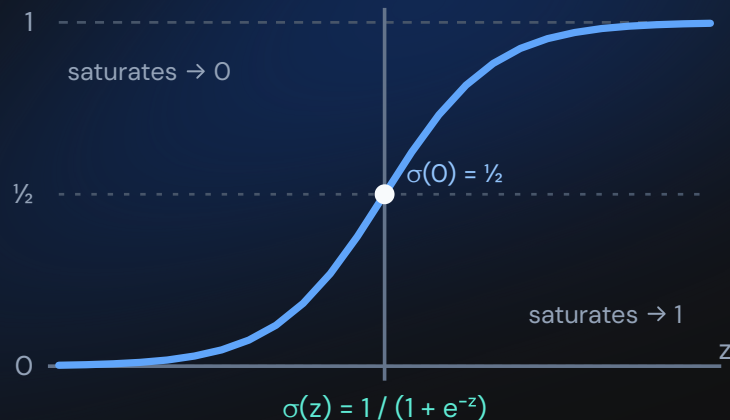
Loss: log loss / cross-entropy

$$\ell(\theta) = -\frac{1}{n} \sum_{i=1}^n \left[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right]$$

Model: sigmoid of a linear score

$$\hat{p}_i = \sigma(x_i^\top \theta) = \frac{1}{1 + e^{-x_i^\top \theta}}$$

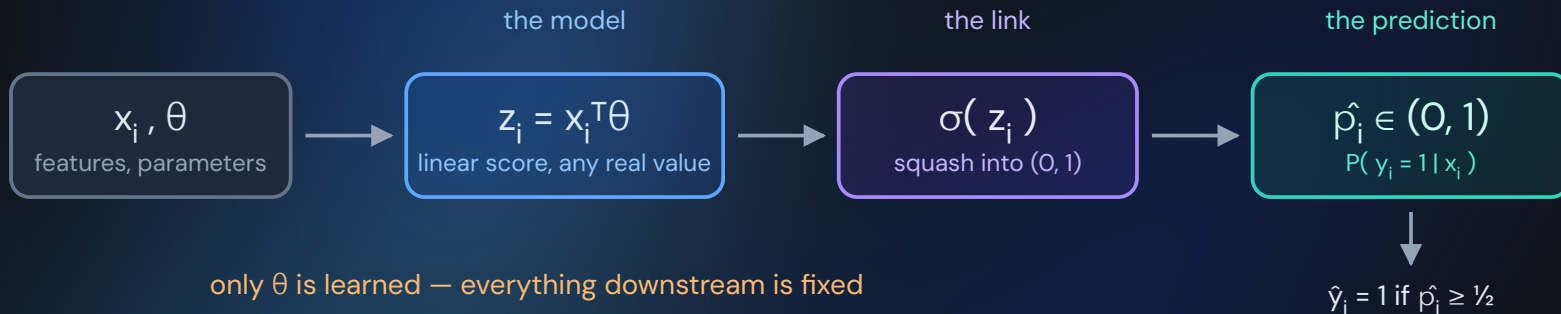
Any real score in, a probability in (0, 1) out. Saturating at both ends, steepest at the decision boundary.



A log of a sigmoid of a dot product. Watch what its gradient turns out to be.

From Score to Probability

Classification is regression's linear score, pushed through one squashing function.



Same skeleton as regression

The linear part $x_i^T \theta$ is identical to the regression model.
Classification only adds σ on the end.

Why we need the gradient of σ

The loss sees $\hat{p}_i$, but we differentiate with respect to θ .
The chain rule has to pass *through* σ .

Deriving the Gradient of $\sigma(x^\top \theta)$

Step 1 – differentiate σ with respect to its own input

$$\begin{aligned}\sigma(z) &= (1 + e^{-z})^{-1} \\ \sigma'(z) &= -(1 + e^{-z})^{-2} \cdot (-e^{-z}) = \frac{e^{-z}}{(1 + e^{-z})^2} \\ &= \underbrace{\frac{1}{1 + e^{-z}}}_{\sigma(z)} \cdot \underbrace{\frac{e^{-z}}{1 + e^{-z}}}_{1 - \sigma(z)} = \sigma(z)(1 - \sigma(z))\end{aligned}$$

Step 2 – differentiate the score with respect to θ_j

$$z_i = \sum_m x_{im} \theta_m \implies \frac{\partial z_i}{\partial \theta_j} = x_{ij}$$

Step 3 – chain them together

$$\begin{aligned}\frac{\partial \hat{p}_i}{\partial \theta_j} &= \frac{\partial}{\partial \theta_j} \sigma(x_i^\top \theta) = \sigma'(z_i) \cdot \frac{\partial z_i}{\partial \theta_j} \\ &= \hat{p}_i (1 - \hat{p}_i) x_{ij}\end{aligned}$$

Stacked over all k parameters

$$\nabla_{\theta} \sigma(x_i^\top \theta) = \hat{p}_i (1 - \hat{p}_i) x_i$$

No minus sign here. MSE carried one from the residual $y_i - x_i^\top \theta$; here we differentiate the prediction itself.

Chaining Through the Log Loss · The Cancellation

Step 1 — chain rule, with $\hat{p}_i$ as the inner function

$$\frac{\partial \ell}{\partial \theta_j} = -\frac{1}{n} \sum_{i=1}^n \left[\frac{y_i}{\hat{p}_i} - \frac{1 - y_i}{1 - \hat{p}_i} \right] \frac{\partial \hat{p}_i}{\partial \theta_j}$$

Step 2 — put the bracket over one denominator

$$\begin{aligned} \frac{y_i}{\hat{p}_i} - \frac{1 - y_i}{1 - \hat{p}_i} &= \frac{y_i(1 - \hat{p}_i) - (1 - y_i)\hat{p}_i}{\hat{p}_i(1 - \hat{p}_i)} \\ &= \frac{y_i - \hat{p}_i}{\hat{p}_i(1 - \hat{p}_i)} \end{aligned}$$

Step 3 — substitute the previous slide's result and cancel

$$\begin{aligned} \frac{\partial \ell}{\partial \theta_j} &= -\frac{1}{n} \sum_i \frac{y_i - \hat{p}_i}{\cancel{\hat{p}_i(1 - \hat{p}_i)}} \cdot \cancel{\hat{p}_i(1 - \hat{p}_i)} x_{ij} \\ &= \frac{1}{n} \sum_i (y_i - \hat{p}_i) x_{ij} \end{aligned}$$

Step 4 — stack the k partials into a vector

$$\nabla_{\theta} \ell(\theta) = \frac{1}{n} X^{\top} (\hat{p} - y)$$

Where the minus went. The loss's leading $-\frac{1}{n}$ flips $y_i - \hat{p}_i$ into $\hat{p}_i - y_i$ — prediction minus target, as in MSE.

Logistic Gradient · The Clean Result

$$\nabla_{\theta} \ell(\theta) = \frac{1}{n} X^{\top} (\hat{p} - y)$$

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def log_loss_grad(theta, X, y):
    n = X.shape[0]
    p_hat = sigmoid(X @ theta)
    return (1 / n) * X.T @ (p_hat - y)
```

MSE

(2/n) ·

$$X^{\top} (X\theta - y)$$

⋮

Log

(1/n) ·

$$X^{\top} (\hat{p} - y)$$

$X^{\top} (\text{prediction} - \text{target})$

only the prediction term differs

The Three Gradients Share One Skeleton

$$\nabla_{\theta} \ell(\theta) \propto \sum_{i=1}^n \underbrace{g(\text{residual}_i)}_{\text{what changes}} \cdot \underbrace{x_i}_{\text{always}}$$

MSE

weights by the **size** of the residual
large errors pull hardest

MAE

weights by the **direction** only
every error pulls equally

Logistic

weights by **how wrong** the
probability was
confidently wrong pulls hardest

Choosing a loss *is* choosing how errors get weighted in every single gradient step.

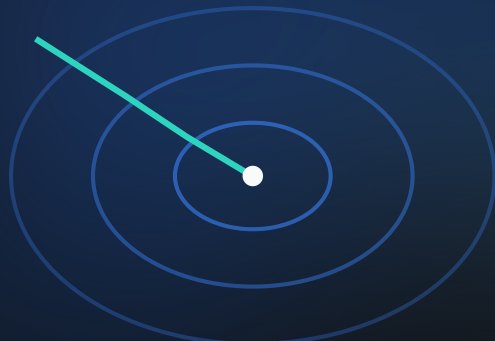
Comparison Table

Loss	Formula	Gradient	Differentiable everywhere?	Weights errors by
MSE	$(1/n) \sum (y_i - x_i^T \theta)^2$	$(2/n) X^T (X\theta - y)$	Yes — smooth	residual magnitude
MAE	$(1/n) \sum y_i - x_i^T \theta $	$-(1/n) X^T \text{sign}(y - X\theta)$	No — corner at 0	residual sign only
Logistic	$-(1/n) \sum [y \log \hat{p} + (1-y) \log(1-\hat{p})]$	$(1/n) X^T (\hat{p} - y)$	Yes — smooth	probability error

Same update rule in all three rows. Only the third column ever changes.

Batch, Stochastic, and Mini-Batch

Same update rule. Only *how much data* goes into each gradient estimate changes.



Batch · all n points

Exact gradient, smooth path, most expensive step.



SGD · one point

Noisy estimate, jagged path, cheapest possible step.



Mini-batch · a subset

The practical middle ground, used almost universally at scale.

Summary

Minimization became an algorithm

Gradient descent turns “minimize the loss” into something concrete, iterative, and implementable in seven lines.

One update rule for every model

$\theta \leftarrow \theta - \eta \nabla L(\theta)$ is the same for every model in this course. Only the gradient formula changes.

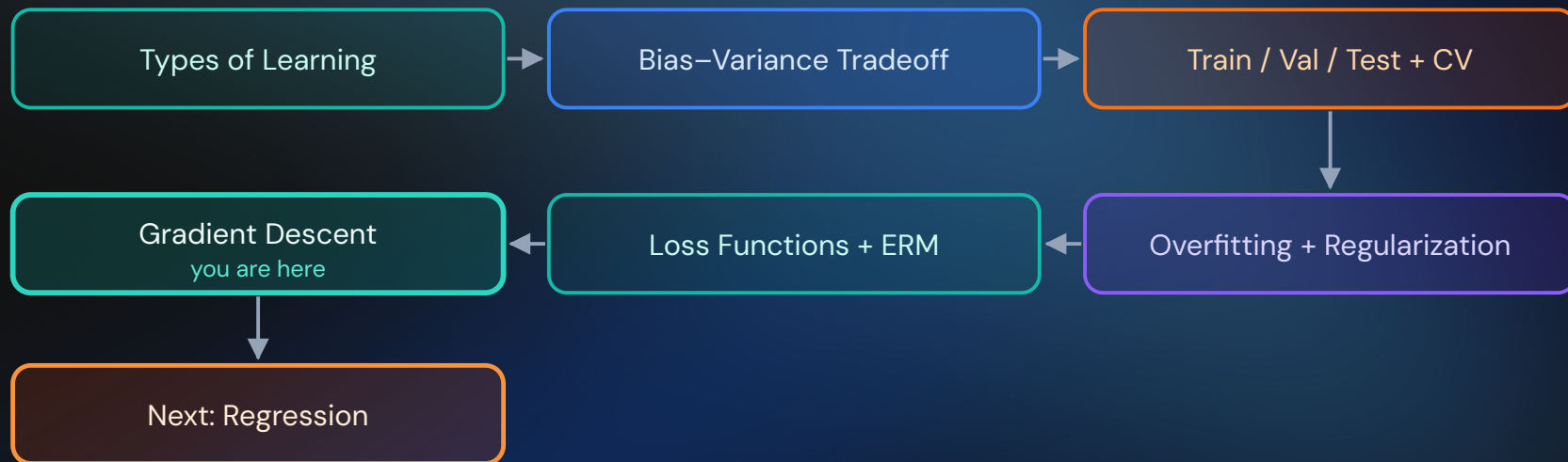
Three losses, three code-ready gradients

MSE, MAE, and logistic loss each reduce to a clean formula — and each weights errors differently.

The unit is complete

Every foundational concept needed to understand *any* supervised learning algorithm is now in place.

Core ML Concepts, Complete



Next unit: **Supervised Learning — Regression**

The MSE gradient derived today reappears verbatim as the training loop.